

Revisions of Ryan and Gerardo's SRS documents by Logan and Tyler

UVSim Software Requirement Specifications

Functional Requirements:

1. The system should take a file provided by a user for data.
2. The system shall parse the file information into a class memory.
3. The system shall execute instructions sequentially within a loop in ascending memory-location order, incrementing the program counter by one to track which step is next to execute.
4. The system should have a case switch that will pick different functions based on the imported files instruction operation.
5. The system shall terminate the program execution when it executes a HALT instruction
6. When the system executes a READ instruction, it shall prompt the user for a word and store the entered word at the specific operand memory location given by the instruction file
7. When the system executes a WRITE instruction, it shall display the word stored at the specific operand memory location given by the instruction file
8. The system shall truncate words to be the last four digits and last three digits if the number is negative of a given word in the case of an overflow.
9. Inputs will be tested for a valid range between 9999 and -999 when the user provides an input.

10. The system shall re-prompt the user if the input is invalid until the user enters a valid input or terminates the program. Valid inputs should be within the given range and be an integer.
11. The system shall verify valid inputs before executing a function.
12. When the system executes a LOAD instruction, it shall copy the word at the operand memory location into the accumulator
13. When the system executes a STORE instruction, it shall copy the accumulator value into the operand memory location
14. When the system executes a SUBTRACT instruction, it shall subtract the word at the operand memory location from the accumulator
15. When the system executes an ADD instruction, it shall add the word at the operand memory location to the accumulator
16. When the system executes a MULTIPLY instruction, it shall multiply the accumulator by the word at the operand memory location
17. When the system executes a DIVIDE instruction, it shall divide the accumulator by the word at the operand memory location
18. When the system executes a BRANCH instruction, it shall set the program counter to the operand memory location
19. The system shall display an error message and stop execution when a DIVIDE instruction attempts to divide by zero
20. The system shall display an error message when it encounters an instruction whose opcode is not defined in BasicML

21. The system shall display an error message and not run when the user selects a file that does not exist
22. The Control operators will function separate of the class functions.
23. System will have unit tests that fail and succeed in knowing different error messages.
24. System will have 100 memory locations that can be accessed in a memory class.

Non-Functional Requirements:

1. Every interactive control in the GUI shall display a text label, identifying its function.
2. The system shall visually indicate whether a program is currently running or idle.
3. The system shall run on Windows, macOS, and Linux without modification to its source code.
4. Users must need feedback if the system hard crashes.
5. Must have a UI that is well-labeled.
6. System will be able to run and execute provided file in 2 seconds with valid inputs.
7. The system will prevent users from accessing private system information.